Applied Economics

Introduction

Pello Aspuru

Winter 2026/2027

Welcome to Applied Economics

All course material (including these slides) will be distributed via **moodle**.

Prior knowledge at the level of Introductory Econometrics (BA) will be assumed throughout the course.

Class participation is highly encouraged, determines 10% of final grade.

Office hours: by email appointment via pello.aspuru@univie.ac.at

People who signed up online for this class and signed the list today are considered **enrolled**.

If you are registered, you will receive a grade!

Welcome to Applied Economics

All course material (including these slides) will be distributed via **moodle**.

Prior knowledge at the level of Introductory Econometrics (BA) will be assumed throughout the course.

Class participation is highly encouraged, determines 10% of final grade.

Office hours: by email appointment via pello.aspuru@univie.ac.at

People who signed up online for this class and signed the list today are considered **enrolled**.

If you are registered, you will receive a grade!

Course Overview

In this Course You Will Learn to...

- Use econometric tools to produce empirical analyses.
- Learn to interpret, and critically evaluate existing empirical studies.
- Understand possibilities and limitations of empirical analyses.

Course Tools

Statistical Software: R and RStudio

- Freely available open source software
- Active user community providing up-to-date functions & support
- Extensive statistical modeling capabilities
- Relevant for industry jobs

Auxiliary Software

- Large Language Models (ChatGPT, Claude etc.) to support learning & coding.
- Latex for drafting academic-style documents, problem set.

Course Tools

Statistical Software: R and RStudio

- Freely available open source software
- Active user community providing up-to-date functions & support
- Extensive statistical modeling capabilities
- Relevant for industry jobs

Auxiliary Software

- Large Language Models (ChatGPT, Claude etc.) to support learning & coding.
- Latex for drafting academic-style documents, problem set.

Course Assessment

Class Participation (10%)

Classroom engagement through questions & comments is encouraged!

Midterm Exam (25%)

Preliminary date: November 25th/26th 2026.

Project Presentations (25%)

- Work on the project in groups of up to 3 students, fill in names using the link provided by Moodle.
- Presentation I (10%): December 16th/17th 2026.
- Presentation II (15%): January 20th/21st 2027.

Final exam (40%)

Date: Wednesday, 27th January 2027.

Course Assessment

Class Participation (10%)

Classroom engagement through questions & comments is encouraged!

Midterm Exam (25%)

Preliminary date: November 25th/26th 2026.

Project Presentations (25%)

- Work on the project in groups of up to 3 students, fill in names using the link provided by Moodle.
- Presentation I (10%): December 16th/17th 2026.
- Presentation II (15%): January 20th/21st 2027.

Final exam (40%)

Date: Wednesday, 27th January 2027.

Useful Textbooks

1. **Heiss, F. (2020).** Using R for Introductory Econometrics (2nd edition)
   available online: http://urfie.net/readOnline.html
2. **Hanck, C., Arnold, M., Gerber, A., & Schmelzer, M. (2023).** Introduction to Econometrics with R
   available online: https://www.econometrics-with-r.org/index.html
3. **Oswald, F., Viers, V., Villedieu, P., & Kennedy, G. (2020).** Introduction to Econometrics with R. Sciences Po, Department of Economics, Paris, France.
   available online: https://scpoecon.github.io/ScPoEconometrics/.

If you need to read up on theoretical concepts:

- Angrist, J. D., & Pischke, J.-S. (2008). Mostly Harmless Econometrics.
- Wooldridge, J. (2015). Introductory Econometrics: A Modern Approach.

What is Applied Economics?

The use of economic theory and **data** to answer real-world **economic** questions.

- Test what economic theory predicts.
  - Theory: a higher minimum wage reduces employment. Do we see this in the data?
- Discover (causal) relationships between economic variables and quantify them.
  - Does immigration increase unemployment for locals? By how much?
- Evaluate and design policies that achieve the intended objective at the **lowest possible cost.**
  - How much money do we need to subsidize public transport to save 1 ton of carbon emissions?

1. Gather, clean and combine multiple sources of data $\rightarrow$ Tidy dataset.
2. Find patterns in the data $\rightarrow$ Data Visualization and Descriptive Statistics.
3. Apply econometric tools $\rightarrow$ Establish causality and estimate magnitude.

What is Applied Economics?

The use of economic theory and **data** to answer real-world **economic** questions.

- Test what economic theory predicts.
  - Theory: a higher minimum wage reduces employment. Do we see this in the data?
- Discover (causal) relationships between economic variables and quantify them.
  - Does immigration increase unemployment for locals? By how much?
- Evaluate and design policies that achieve the intended objective at the **lowest possible cost.**
  - How much money do we need to subsidize public transport to save 1 ton of carbon emissions?

1. Gather, clean and combine multiple sources of data → Tidy dataset.
2. Find patterns in the data → Data Visualization and Descriptive Statistics.
3. Apply econometric tools → Establish causality and estimate magnitude.

What is Applied Economics?

The use of economic theory and **data** to answer real-world **economic** questions.

- Test what economic theory predicts.
  - Theory: a higher minimum wage reduces employment. Do we see this in the data?
- Discover (causal) relationships between economic variables and quantify them.
  - Does immigration increase unemployment for locals? By how much?
- Evaluate and design policies that achieve the intended objective at the **lowest possible cost.**
  - How much money do we need to subsidize public transport to save 1 ton of carbon emissions?

1. Gather, clean and combine multiple sources of data → Tidy dataset.
2. Find patterns in the data → Data Visualization and Descriptive Statistics.
3. Apply econometric tools → Establish causality and estimate magnitude.

What is Applied Economics?

The use of economic theory and **data** to answer real-world **economic** questions.

- Test what economic theory predicts.
  - Theory: a higher minimum wage reduces employment. Do we see this in the data?
- Discover (causal) relationships between economic variables and quantify them.
  - Does immigration increase unemployment for locals? By how much?
- Evaluate and design policies that achieve the intended objective at the **lowest possible cost.**
  - How much money do we need to subsidize public transport to save 1 ton of carbon emissions?

1. Gather, clean and combine multiple sources of data → Tidy dataset.
2. Find patterns in the data → Data Visualization and Descriptive Statistics.
3. Apply econometric tools → Establish causality and estimate magnitude.

What is Applied Economics?

The use of economic theory and **data** to answer real-world **economic** questions.

- Test what economic theory predicts.
  - Theory: a higher minimum wage reduces employment. Do we see this in the data?
- Discover (causal) relationships between economic variables and quantify them.
  - Does immigration increase unemployment for locals? By how much?
- Evaluate and design policies that achieve the intended objective at the **lowest possible cost.**
  - How much money do we need to subsidize public transport to save 1 ton of carbon emissions?

1. Gather, clean and combine multiple sources of data $\rightarrow$ Tidy dataset.
2. Find patterns in the data $\rightarrow$ Data Visualization and Descriptive Statistics.
3. Apply econometric tools $\rightarrow$ Establish causality and estimate magnitude.

What is R ?

- **Programming Language** A formal system of rules and symbols that allows humans to write instructions a computer can understand and execute.
  - Perfect replicability.
- **R** is a programming language and software environment for...
  - Data tidying.
  - Data visualization.
  - Statistical and econometric analysis.
- Install R from https://cran.r-project.org/

Basic R Programming Concepts

Code instructions written in a programming language.

- **Script** a .R text file with many lines of code.
- **Comment** code lines ignored by the computer, used to explain or annotate your work. Comments start with #.
- **Object** a piece of information (like a number, or dataset) stored in memory that can be used and modified by your code.
- **Function** a reusable "recipe" that takes some objects, performs a specific task and returns an output.

```r
# This is a comment

# Create object "n"
n <- 100

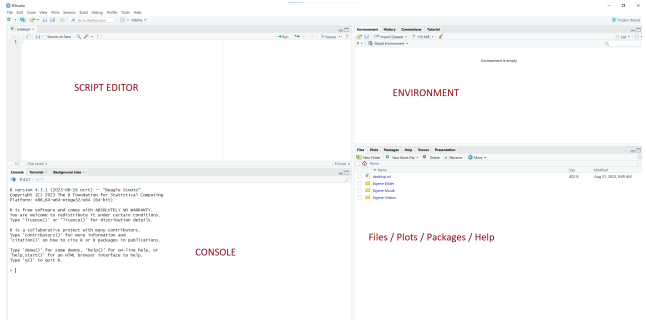
# Define the function "square()"
square <- function(x) { return(x^2) }

# Run function on object, store as "result"
result <- square(n)

# Print the result in the console
print(result)
```

What is RStudio?

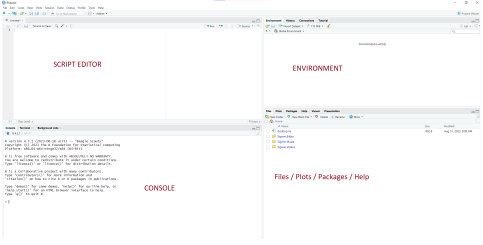
- **RStudio** a workspace that provides a user-friendly interface to write and run code.
- Install RStudio from https://posit.co/download/rstudio-desktop/
- R Studio interface consists of 4 main windows (panes):



RStudio Interface

Console

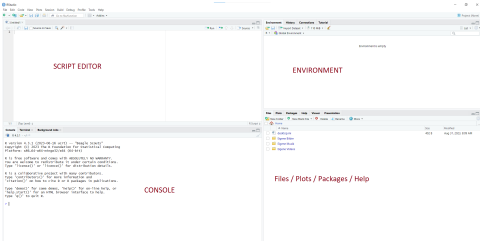
- Output of your coding and computations appears here (exception: graphs).
- It is possible to write code directly into the console.
- Do not use the console to make changes to the data.



RStudio Interface

Script Editor

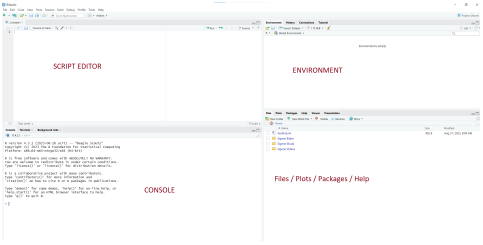
- Open a new R script: click *File* → *New File* → *R Script*.
- Always write your main code in a script: allows you to save and keep track of your work, and to re-run the analysis or make modifications later.
- To send the code in your script to the console, click *run*.



RStudio Interface

Environment

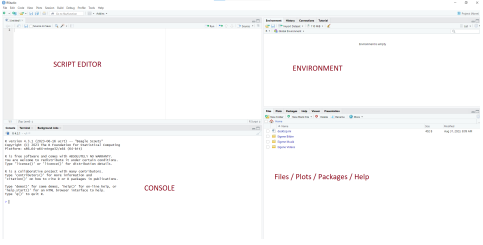
- list of all objects you created
- second pane provides *History* of commands entered into the console



RStudio Interface

Files / Plots / Packages / Help / Viewer window

- *Plots* you create appear here.
- if you search for *Help*, information appears here.



Script Set Up

Working directory is the default folder on your computer where R reads and saves files, unless you tell it otherwise.

- Comments start with #: R ignores them.
- Create a new folder on your desktop called "AE_2026" to use as your working directory.
- show current working directory `getwd()`
- setting new working directory to newly created folder `setwd("C:/Users/.../AE_2026")`
- list all files in your working directory `dir()`

```
# This is a comment

# Check current working directory
getwd()

# Set a new working directory.
setwd("C:/Users/YourName/Desktop/AE_2026")

# Verify the change
getwd()

# List files in the working directory
dir()
```

Script Set Up

Working directory is the default folder on your computer where R reads and saves files, unless you tell it otherwise.

- Comments start with #: R ignores them.
- Create a new folder on your desktop called "AE_2026" to use as your working directory.
- show current working directory `getwd()`
- setting new working directory to newly created folder `setwd("C:/Users/.../AE_2026")`
- list all files in your working directory `dir()`

```
# This is a comment

# Check current working directory
getwd()

# Set a new working directory.
setwd("C:/Users/YourName/Desktop/AE_2026")

# Verify the change
getwd()

# List files in the working directory
dir()
```

Clean the Environment

- Every object you create stays in the **environment** until you remove it.
- Start every script from a clean environment, so that your results do not depend on objects left over from earlier work.
- list all objects: `ls()`; remove all of them: `rm(list = ls())`

```
# List all objects in the environment
ls()

# Remove all objects from the environment
rm(list = ls())

# Verify the change
ls()
```

R Objects

Object a piece of information (like a number, word, vector, or dataset) stored in memory that can be used and modified by your code.

- create an object: `object <- ...`
- print object to the console: `object`
- identify format using `str(object)`
- get basic information on the object: `class(object)`, `dim(object)`
- list all objects: `ls()`
- remove an object: `rm(object)`

```r
# Create object "n"
n <- 100

# Print the object in the console:
n

# generate vector "age":
age <- sample(18:65, n, replace = TRUE)

# Visualize
age # Print object in console
str(age) # Object type, size
class(age) # Object type
dim(age)
ls() # List all objects
```

R Objects

Object a piece of information (like a number, word, vector, or dataset) stored in memory that can be used and modified by your code.

- create an object: `object <- ...`
- print object to the console: `object`
- identify format using `str(object)`
- get basic information on the object: `class(object)`, `dim(object)`
- list all objects: `ls()`
- remove an object: `rm(object)`

```
# Create object "n"
n <- 100

# Print the object in the console:
n

# generate vector "age":
age <- sample(18:65, n, replace = TRUE)

# Visualize
age # Print object in console
str(age) # Object type, size
class(age) # Object type
dim(age)
ls() # List all objects
```

Vectors

Vector an ordered sequence of values of the same type (all numbers, or all words).

- `AirPassengers`: built-in dataset with the number of airline passengers (in thousands) in each month from 1949 to 1960.
- number of elements: `length(vector)`
- access elements by their position, in square brackets: `vector[...]`

```
# Three vectors with one value per month
passengers <- as.numeric(AirPassengers)
year       <- as.numeric(floor(time(AirPassengers)))
month      <- as.numeric(cycle(AirPassengers))

length(passengers)     # 144

# Access elements of a vector
passengers[1]          # first element: 112
passengers[1:3]        # elements 1 to 3: 112 118 132
passengers[c(1, 144)]  # first and last: 112 432
```

Vectors

Vector an ordered sequence of values of the same type (all numbers, or all words).

- `AirPassengers`: built-in dataset with the number of airline passengers (in thousands) in each month from 1949 to 1960.
- number of elements: `length(vector)`
- access elements by their position, in square brackets: `vector[...]`

```
# Three vectors with one value per month
passengers <- as.numeric(AirPassengers)
year       <- as.numeric(floor(time(AirPassengers)))
month      <- as.numeric(cycle(AirPassengers))

length(passengers)     # 144

# Access elements of a vector
passengers[1]          # first element: 112
passengers[1:3]        # elements 1 to 3: 112 118 132
passengers[c(1, 144)]  # first and last: 112 432
```

From Vectors to Dataframes

Dataframes are two-dimensional objects that store data in a tabular format, with rows and columns:

- Rows: units of observation (individuals, years, ...).
- Columns: variables (e.g. income, education, GNP, ...) [vectors]

`data.frame()` puts vectors of the same length side by side as columns; `head()` prints the first rows:

```
# Create a dataset from the three vectors
df_air <- data.frame(year, month, passengers)
dim(df_air)             # 144 rows, 3 columns
```

```
> head(df_air)
  year month passengers
1 1949     1        112
2 1949     2        118
3 1949     3        132
4 1949     4        129
5 1949     5        121
6 1949     6        135
```

a column is a **variable**: the vector passengers

a row is an **observation**: March 1949

	year	month	passengers
1	1949	1	112
2	1949	2	118
3	1949	3	132
4	1949	4	129
5	1949	5	121
6	1949	6	135

From Vectors to Dataframes

Dataframes are two-dimensional objects that store data in a tabular format, with rows and columns:

- Rows: units of observation (individuals, years, ...).
- Columns: variables (e.g. income, education, GNP, ...) [vectors]

`data.frame()` puts vectors of the same length side by side as columns; `head()` prints the first rows:

```
# Create a dataset from the three vectors
df_air <- data.frame(year, month, passengers)
dim(df_air)             # 144 rows, 3 columns
```

```
> head(df_air)
  year month passengers
1 1949     1        112
2 1949     2        118
3 1949     3        132
4 1949     4        129
5 1949     5        121
6 1949     6        135
```

a column is a **variable**: the vector passengers

	year	month	passengers
1	1949	1	112
2	1949	2	118
3	1949	3	132
4	1949	4	129
5	1949	5	121
6	1949	6	135

a row is an **observation**: March 1949

Functions

- A **function** takes one or more objects as input, performs a task, and returns an output.
- We have already used several, such as `sample()`, `length()` and `data.frame()`. We can also write our own:

```
ls()

# Define the function "square()"
square <- function(x) { return(x^2) }

# Run function on object, store as "result"
result <- square(n)

# Print the result in the console
print(result)

ls()
```

R Packages

What are packages?

- Collections of functions, data, and documentation that extend R's functionality.
- Thousands are available on CRAN, the official R package repository.
- Example: `dplyr` to manipulate dataframes, `ggplot2` to draw graphs.

How to install a package (once per computer)

```
# Install once (downloads from CRAN)
install.packages("dplyr")
```

How to load a package (every time you start R)

```
# Load every time you start R
library(dplyr)
```

Size and Structure

Before running any analysis, take a first look at the data: how large it is, which variables it contains, what values they take. This helps to spot mistakes early.

- number of rows and columns: `dim()`
- variable names: `names()`
- type of each variable and its first values: `str()`
- first / last six rows: `head()`, `tail()`

```
dim(df_air)
names(df_air)
str(df_air)

tail(df_air)
```

Summary Statistics and Frequency Tables

- `summary()`: minimum, quartiles, mean and maximum of a numerical variable.
- Applied to a whole dataframe, it summarises every variable at once.
- Individual statistics: `mean()`, `median()`, `var()`, `sd()`.
- `table()` counts how many observations take each value of a variable: most useful for categorical variables.

```
summary(df_air$passengers)

summary(df_air)

mean(df_air$passengers)
sd(df_air$passengers)

table(df_air$year)
```

Selecting Observations and Variables

Square brackets select parts of a dataframe by position: `dataframe[rows, columns]`. Leaving rows or columns empty selects all of them. The $ sign selects a single variable by its name.

```
# Accessing data through row/column indexes
df_air[1:3,] # Rows 1 to 3, all columns.
df_air[c(2,5),c(1,3)] # Rows 2 and 5, Columns 1 and 3.
head(df_air[,c(1,3)]) # All rows, Columns 1 and 3 (first six rows shown).

# Access the 'passengers' variable.
df_air$passengers
```

Store a subset under a **new name**, which keeps the original data. Danger! `df_air <- df_air[...,...]` drops everything that is not selected.

```
df_air_first_9_rows <- df_air[1:9,]
ls()
```

Selecting Observations and Variables

Square brackets select parts of a dataframe by position: `dataframe[rows, columns]`. Leaving rows or columns empty selects all of them. The $ sign selects a single variable by its name.

```
# Accessing data through row/column indexes
df_air[1:3,] # Rows 1 to 3, all columns.
df_air[c(2,5),c(1,3)] # Rows 2 and 5, Columns 1 and 3.
head(df_air[,c(1,3)]) # All rows, Columns 1 and 3 (first six rows shown).

# Access the 'passengers' variable.
df_air$passengers
```

Store a subset under a **new name**, which keeps the original data. Danger! `df_air <- df_air[...,...]` drops everything that is not selected.

```
df_air_first_9_rows <- df_air[1:9,]
ls()
```

From Existing Variables

- A new variable is created by assigning a vector to a name that does not yet exist in the dataframe: `dataframe$new_variable <- ...`
- The vector is usually computed from existing variables.
- `passengers` is measured in thousands, so dividing it by 1000 gives the number of passengers in millions:

```
df_air$passengers_millions <- df_air$passengers / 1000
head(df_air)
```

Logical Conditions

A logical condition compares values and returns `TRUE` or `FALSE` for each element of a vector.

Operator	Meaning
==	equal to
!=	not equal to
>, >=	greater than (or equal to)
<, <=	less than (or equal to)
&	and: both are true
|	or: at least one is true
%in%	one of a set of values

The following condition is `TRUE` for the months between 1950 and 1959; `table()` counts how many observations satisfy it:

```
# Logical condition vector
index_fifties <- (df_air$year >= 1950) & (df_air$year <= 1959)
table(index_fifties)
```

Logical Conditions

A logical condition compares values and returns `TRUE` or `FALSE` for each element of a vector.

Operator	Meaning
==	equal to
!=	not equal to
>, >=	greater than (or equal to)
<, <=	less than (or equal to)
&	and: both are true
|	or: at least one is true
%in%	one of a set of values

The following condition is `TRUE` for the months between 1950 and 1959; `table()` counts how many observations satisfy it:

```
# Logical condition vector
index_fifties <- (df_air$year >= 1950) & (df_air$year <= 1959)
table(index_fifties)
```

Variables Based on a Condition

- `ifelse(condition, a, b)` returns `a` where the condition is `TRUE` and `b` where it is `FALSE`: useful for creating categorical variables.
- `season` separates the summer months from the rest of the year.
- `traffic` separates the months with more passengers than the median month from the others.

```
df_air$season  <- ifelse(df_air$month %in% 6:8, "Summer", "Rest of year")
df_air$traffic <- ifelse(df_air$passengers > median(df_air$passengers), "High", "Low")

table(df_air$season)
```

Two-way Tables and Selecting with a Condition

With two variables, `table()` returns a **two-way table**, which counts the observations for each combination of categories. `prop.table(..., margin = 1)` turns the counts into shares within each row:

```
table(df_air$season, df_air$traffic)

prop.table(table(df_air$season, df_air$traffic), margin = 1)
```

A logical vector selects the rows where it is `TRUE`:

```
df_air_fifties <- df_air[index_fifties, ]
dim(df_air_fifties)
```

Two-way Tables and Selecting with a Condition

With two variables, `table()` returns a **two-way table**, which counts the observations for each combination of categories. `prop.table(..., margin = 1)` turns the counts into shares within each row:

```
table(df_air$season, df_air$traffic)

prop.table(table(df_air$season, df_air$traffic), margin = 1)
```

A logical vector selects the rows where it is `TRUE`:

```
df_air_fifties <- df_air[index_fifties, ]
dim(df_air_fifties)
```

A Second Dataset: `longley`

- R is able to work with multiple datasets at the same time.
- `longley`: built-in dataset with yearly US macroeconomic data, 1947–1962.

```
# longley: US macroeconomic data, one row per year (1947-1962)
data(longley)
class(longley)

dim(longley)
head(longley)
```

```
> head(longley)
     GNP.deflator     GNP Unemployed Armed.Forces Population Year Employed
1947         83.0 234.289      235.6         159.0    107.608 1947   60.323
1948         88.5 259.426      232.5         145.6    108.632 1948   61.122
1949         88.2 258.054      368.2         161.6    109.773 1949   60.171
1950         89.5 284.599      335.1         165.0    110.929 1950   61.187
1951         96.2 328.975      209.9         309.9    112.075 1951   63.221
1952         98.1 346.999      193.2         359.4    113.270 1952   63.639
```

- Each **row** is one year: the unit of observation is the US economy in a given year.
- Each **column** is a variable: here, all seven are numerical.
- The first column has no name: it shows the **row names**, not a variable.

A Second Dataset: `longley`

- R is able to work with multiple datasets at the same time.
- `longley`: built-in dataset with yearly US macroeconomic data, 1947–1962.

```
# longley: US macroeconomic data, one row per year (1947-1962)
data(longley)
class(longley)

dim(longley)
head(longley)
```

```
> head(longley)
     GNP.deflator     GNP Unemployed Armed.Forces Population Year Employed
1947         83.0 234.289      235.6         159.0    107.608 1947   60.323
1948         88.5 259.426      232.5         145.6    108.632 1948   61.122
1949         88.2 258.054      368.2         161.6    109.773 1949   60.171
1950         89.5 284.599      335.1         165.0    110.929 1950   61.187
1951         96.2 328.975      209.9         309.9    112.075 1951   63.221
1952         98.1 346.999      193.2         359.4    113.270 1952   63.639
```

- Each **row** is one year: the unit of observation is the US economy in a given year.
- Each **column** is a variable: here, all seven are numerical.
- The first column has no name: it shows the **row names**, not a variable.

The `longley` Variables

Variable	Description	Units
`GNP.deflator`	GNP implicit price deflator (price level)	index, 1954 = 100
`GNP`	Gross National Product	billions of current US dollars
`Unemployed`	number of unemployed people	tens of thousands of people
`Armed.Forces`	number of people in the armed forces	tens of thousands of people
`Population`	non-institutionalized population aged 14+	millions of people
`Year`	the year of the observation	calendar year
`Employed`	number of employed people	millions of people

Example: in 1947, `Unemployed` is 235.6, about 2.36 million unemployed people, and `Employed` is 60.323, about 60.3 million employed people.

The `longley` Variables

Variable	Description	Units
`GNP.deflator`	GNP implicit price deflator (price level)	index, 1954 = 100
`GNP`	Gross National Product	billions of current US dollars
`Unemployed`	number of unemployed people	tens of thousands of people
`Armed.Forces`	number of people in the armed forces	tens of thousands of people
`Population`	non-institutionalized population aged 14+	millions of people
`Year`	the year of the observation	calendar year
`Employed`	number of employed people	millions of people

Example: in 1947, `Unemployed` is 235.6, about 2.36 million unemployed people, and `Employed` is 60.323, about 60.3 million employed people.

Creating Variables with dplyr

The old-fashioned way:

```
longley$employment_share <- longley$Employed/longley$Population
```

The dplyr way: mutate() creates several variables at once, and the pipe |> passes the dataframe on to the next function.

```
longley <- longley |>
  mutate(employment_share   = Employed / Population,
         unemployment_share = (Unemployed / 100) / Population,
         population_growth  = (Population - lag(Population)) / lag(Population))

head(longley)
```

Creating Variables with dplyr

The old-fashioned way:

```
longley$employment_share <- longley$Employed/longley$Population
```

The dplyr way: mutate() creates several variables at once, and the pipe |> passes the dataframe on to the next function.

```
longley <- longley |>
  mutate(employment_share   = Employed / Population,
         unemployment_share = (Unemployed / 100) / Population,
         population_growth  = (Population - lag(Population)) / lag(Population))

head(longley)
```

Aggregating Data

- To combine `df_air` with `longley`, both datasets need the same frequency: `df_air` is monthly, `longley` is annual.
- We aggregate `df_air` from monthly to annual data: `group_by()` forms one group per year, `summarise()` computes one row per group.

```
head(df_air)
```

```
# Aggregate the monthly data to one row per year
df_air_annual <- df_air |>
  group_by(year) |>
  summarise(annual_passengers        = sum(passengers),
            mean_monthly_passengers = mean(passengers),
            min_monthly_passengers  = min(passengers),
            max_monthly_passengers  = max(passengers))

head(df_air_annual)
```

Merging Datasets

- **Merging** (joining) puts the columns of two datasets side by side, matching the rows that refer to the same observation.
- The variable used to match the rows is the **key**: the year, called `year` in `df_air_annual` and `Year` in `longley`, so we use `by = c("year" = "Year")`.
- Problem: the two datasets do not cover the same years.

```
df_air_annual$year
longley$Year
```

Function	Rows it keeps	Rows in our example
`inner_join()`	only the rows with a match in both datasets	12 (1949 to 1960)
`left_join()`	all rows of the left dataset	12 (1949 to 1960)
`right_join()`	all rows of the right dataset	16 (1947 to 1962)
`full_join()`	all rows of both datasets	16 (1947 to 1962)

Merging Datasets

- **Merging** (joining) puts the columns of two datasets side by side, matching the rows that refer to the same observation.
- The variable used to match the rows is the **key**: the year, called `year` in `df_air_annual` and `Year` in `longley`, so we use `by = c("year" = "Year")`.
- Problem: the two datasets do not cover the same years.

```
df_air_annual$year
longley$Year
```

Function	Rows it keeps	Rows in our example
`inner_join()`	only the rows with a match in both datasets	12 (1949 to 1960)
`left_join()`	all rows of the left dataset	12 (1949 to 1960)
`right_join()`	all rows of the right dataset	16 (1947 to 1962)
`full_join()`	all rows of both datasets	16 (1947 to 1962)

Inner and Left Joins

In `x |> some_join(y)`, `x` is the *left* dataset and `y` the *right* dataset.
`inner_join()` keeps only the years that appear in both datasets:

```
# Merge the two annual datasets, keeping the years present in both
df_inner <- df_air_annual |>
  inner_join(longley, by = c("year" = "Year"))

dim(df_inner)
head(df_inner)
```

`left_join()` keeps every row of `df_air_annual`. Every year in it also appears in `longley`, so the result has the same 12 rows:

```
df_left <- df_air_annual |>
  left_join(longley, by = c("year" = "Year"))

dim(df_left)
```

Inner and Left Joins

In `x |> some_join(y)`, `x` is the *left* dataset and `y` the *right* dataset.
`inner_join()` keeps only the years that appear in both datasets:

```
# Merge the two annual datasets, keeping the years present in both
df_inner <- df_air_annual |>
  inner_join(longley, by = c("year" = "Year"))

dim(df_inner)
head(df_inner)
```

`left_join()` keeps every row of `df_air_annual`. Every year in it also appears in `longley`, so the result has the same 12 rows:

```
df_left <- df_air_annual |>
  left_join(longley, by = c("year" = "Year"))

dim(df_left)
```

Right Joins

`right_join()` keeps every row of `longley`, so the result has 16 rows. 1947, 1948, 1961 and 1962 have no match in `df_air_annual`, so their passenger variables are missing (`NA`):

```
df_right <- df_air_annual |>
  right_join(longley, by = c("year" = "Year"))

dim(df_right)

# Rows without passenger data
df_right[is.na(df_right$annual_passengers), c("year", "annual_passengers", "GNP")]
```

- Which join to use depends on which observations you want to keep.
- A **left join** is the most common choice: it keeps your main dataset intact and adds information from a second one, without dropping or adding rows.

Right Joins

`right_join()` keeps every row of `longley`, so the result has 16 rows. 1947, 1948, 1961 and 1962 have no match in `df_air_annual`, so their passenger variables are missing (`NA`):

```
df_right <- df_air_annual |>
  right_join(longley, by = c("year" = "Year"))

dim(df_right)

# Rows without passenger data
df_right[is.na(df_right$annual_passengers), c("year", "annual_passengers", "GNP")]
```

- Which join to use depends on which observations you want to keep.
- A **left join** is the most common choice: it keeps your main dataset intact and adds information from a second one, without dropping or adding rows.

Missing Values

- `is.na()` returns `TRUE` for every missing value. R counts `TRUE` as 1, so `sum()` gives the number of missing values in a variable, and `colSums()` gives it for every variable.

```
sum(is.na(df_right$annual_passengers))
colSums(is.na(df_right))
```

- Many functions return `NA` if the variable contains missing values. `na.rm = TRUE` tells them to ignore the missing values:

```
mean(df_right$annual_passengers)
mean(df_right$annual_passengers, na.rm = TRUE)
```

Missing Values

- `is.na()` returns `TRUE` for every missing value. R counts `TRUE` as 1, so `sum()` gives the number of missing values in a variable, and `colSums()` gives it for every variable.

```
sum(is.na(df_right$annual_passengers))
colSums(is.na(df_right))
```

- Many functions return `NA` if the variable contains missing values. `na.rm = TRUE` tells them to ignore the missing values:

```
mean(df_right$annual_passengers)
mean(df_right$annual_passengers, na.rm = TRUE)
```